CS 254 Machine Learning
Project Final Report

1. Introduction

In modern textile manufacturing, fabric quality maintenance is a critical step, but detection of defects is typically a serious bottleneck process. Traditionally, this relies on extensive hand inspection, whereby workers visually inspect cloth for defects, a process both time and prone to human error. This step is thus highly critical to automate, as it has the potential to hugely boost production efficiency and consistency. This project proposes an automated system by way of a machine learning model that detects such production faults directly from images captured by camera systems.

Our overall model utilizes a Convolutional Neural Network (CNN) using the TensorFlow Keras Sequential API. This is the baseline system, chosen for its proven history of application in image classification problems and its ease of implementation by following well-documented TensorFlow tutorials. The model is trained on the MVTec Anomaly Detection (AD) dataset, which is a set of labeled 32x32 fabric patches that are either "good" or "damaged". We have additionally taken the liberty of simplifying the nature of the problem by aggregating five different types of defects into a single "damaged" class.

This project demonstrates the feasibility of using a CNN in textile defect detection. The baseline model at the beginning has good accuracy over training data but has issues with the class imbalance of the data set and has a low F1-score on the minority class. However, using methods such as data augmentation, MLPs, and transfer learning, we were able to build a stronger and more accurate classifier.

2. Problem Definition and Algorithm

2.1 Task Definition

When cloth is produced in factories, a lot of different mishaps might happen. Currently it takes a lot of manual labour to filter out errors in production. Using camera systems to record images of cloth while they are being produced, errors could

be detected automatically through a pretrained model. To train the model, the input consists of labeled images of cloth either in a good or a damaged state. To simplify training time, the images are 64x64 or 32x32 patches.

2.2 Algorithm Definition

The model is a Convolutional Neural Network (CNN) built using Keras Sequential API for image classifications. This CNN serves as a baseline system, and was chosen because of the tensorflow image classification template. The model learns spatial patterns (defects) through convolution and pooling, then classifies using fully connected layers. To improve performance even further, both data augmentation and transfer learning were applied to our base model.

2.3 Problem Definition and Algorithm

2.3.1 Task Definition

When cloth is produced in factories, a lot of different mishaps might happen. Currently it takes a lot of manual labour to filter out errors in production. Using camera systems to record images of cloth while they are being produced, errors could be detected automatically through a pretrained model. To train the model, the input consists of labeled images of cloth either in a good or a damaged state. To simplify training time, the images will be 64x64 or 32x32 patches.

2.3.2 Dataset

The data is based on the dataset "The MVTec anomaly detection dataset (MVTec AD)" by the company MVTec and was prepared by Belkhir for Kaggle. It contains images of fabric split into training and test sets with non-overlapping sources. From the original high-definition images, 64x64 patches are generated. Each patch is further rotated at eight fixed angles from 0 to 140 degrees to support rotation-robust classification.

The data is labeled in 6 categories: good (no defect), color, cut, hole, thread and metal contamination.

In the first version, we convert the 5 defect categories into one category "damaged". Therefore our classification is simplified to show a proof of concept. Later all categories will be used for the final presentation.

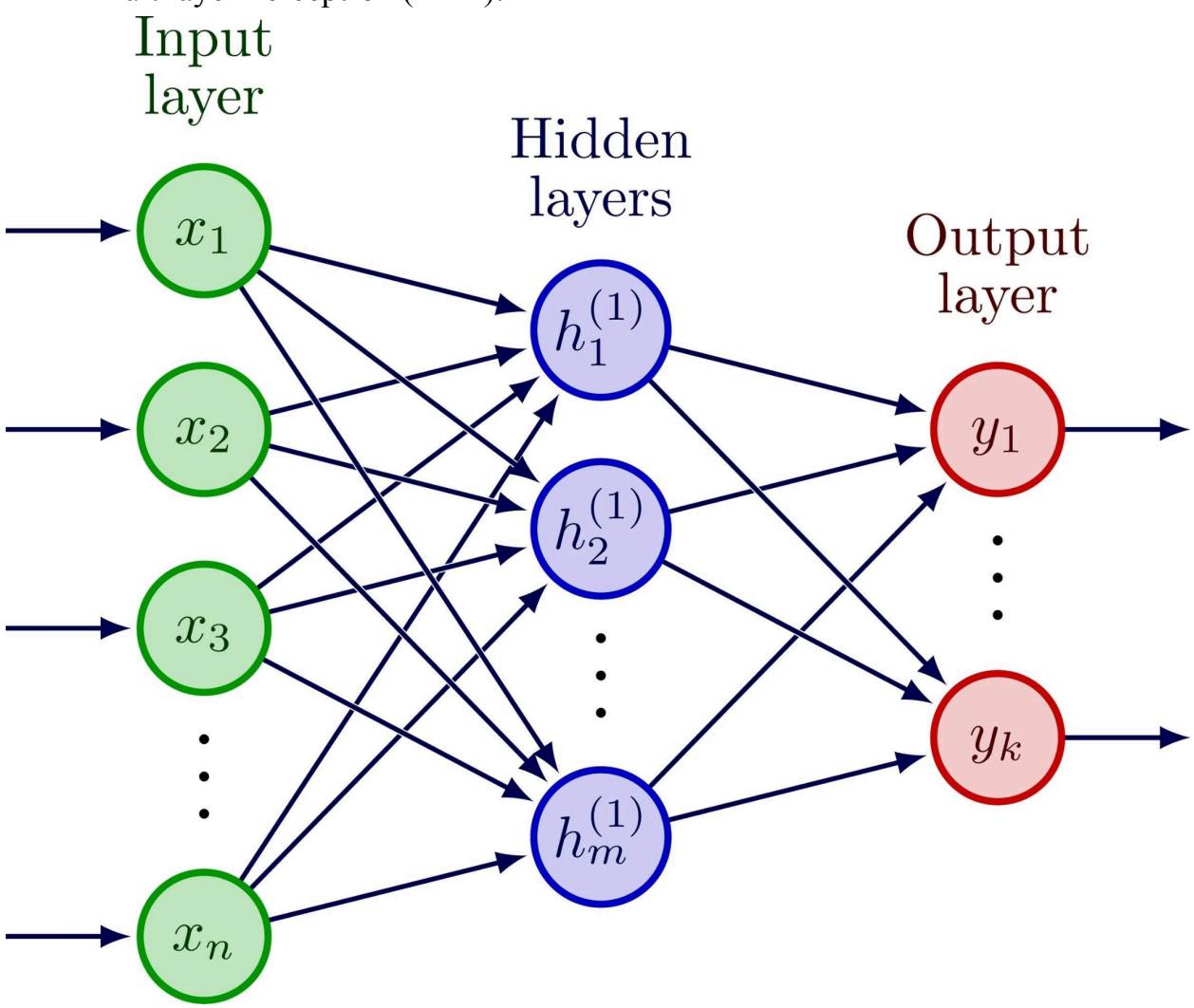
2.3.3 Algorithm Definition

The model is a Convolutional Neural Network (CNN) built using Keras Sequential API for image classifications. This CNN serves as a baseline system, and was chosen because of the tensorflow image classification template. The model learns spatial patterns (defects) through convolution and pooling, then classifies using fully connected layers. Future improvements may include class balancing (class_weight or oversampling), data augmentation, or deeper architectures (e.g., transfer learning).

2.4 Additional models

To further analyze the performance of our baseline CNN and improve our classification results, we explored two additional architectural approaches: Multilayer Perceptrons (MLP) and Transfer Learning.

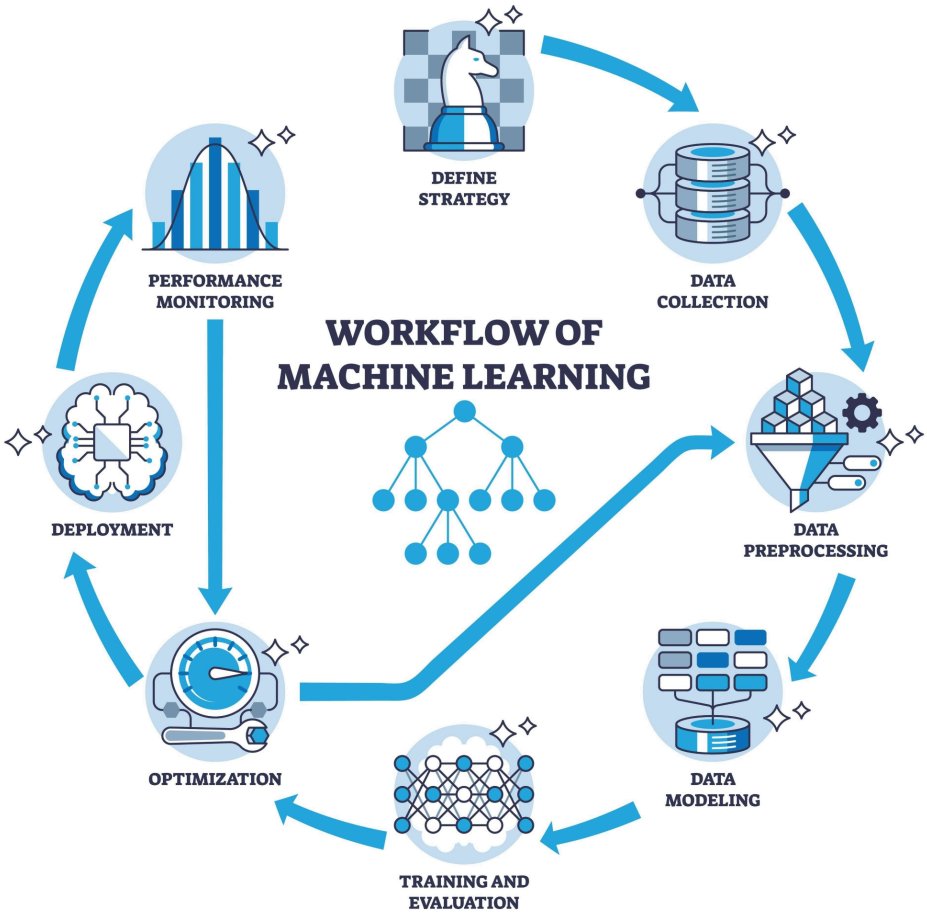
- Multilayer Perceptron (MLP):



The Multilayer Perceptron, MLP, is a basic feedforward artificial neural network. Unlike CNNs that are designed to maintain spatial hierarchies of pictures through convolution, the MLP flattens the input image into a vector and further processes it through fully connected layers-dense layers. A typical MLP would contain an input layer, one or more hidden layers, and an output layer.

Every neuron in the hidden layers uses a nonlinear activation function on the weighted sum of its inputs; thus, the model learns complex patterns. While CNNs are generally superior for image tasks, the implementation of an MLP serves as an essential baseline against which one can contrast the specific value provided by spatial feature extraction in textile defect detection.

- Transfer learning



2.5 MLP Baseline

Besides experimenting with more advanced models, we also tested a simple Multilayer Perceptron on the 32×32 and 64x64 image patches. The MLP flattens the input into a vector and processes it using fully-connected layers without learning any spatial patterns. Because of this, the model collapsed to predicting mostly “damaged,” which is the majority class.

The results showed high overall accuracy but almost no ability to detect the minority “good” samples. This confirmed that MLPs are not suitable for our defect-detection task and mostly serve as a baseline comparison to show why convolutional models are necessary.

32x32 result

```
MLP F1 (binary): 0.0
              precision    recall  f1-score   support

     damaged       0.83      1.00      0.91     30000
        good       0.00      0.00      0.00      6000

    accuracy                           0.83     36000
   macro avg       0.42      0.50      0.45     36000
weighted avg       0.69      0.83      0.76     36000
```

64x64 result

```
MLP F1 (binary): 0.0
              precision    recall  f1-score   support

     damaged       0.83      1.00      0.91     30000
        good       0.00      0.00      0.00      6000

    accuracy                           0.83     36000
   macro avg       0.42      0.50      0.45     36000
weighted avg       0.69      0.83      0.76     36000
```

3. Experimental Evaluation

3.1 Methodology

To evaluate our baseline model, we used common metrics like accuracy, precision, recall and F1-score. Because the dataset is pretty imbalanced (about 81% damaged and 19% good), accuracy alone didn't really show the real performance. So we mostly looked at the F1-score for the "good" class since it's the smaller one and harder to detect.

We expected that a simple CNN trained on the MVTec AD dataset would get high accuracy but have problems with the minority class. To test this, we trained it for 10 epochs on 32x32 fabric images, using 80% for training and 20% for validation. We tracked accuracy and loss per epoch and tested it on a separate test set after.

We think that this way makes sense because it directly checks how well the model tells apart damaged and good fabrics, which is the main goal. We also compared our results with similar CNN models in other works that usually reach around 80–90% accuracy, so it's a fair baseline.

3.2 Results

The first version of our model, using 32x32 images, had a F1 score of 0.475. This is because the images are very small, and there is a high data imbalance, with more damaged than good samples.

```
F1 (binary): 0.47505270555165147
              precision    recall  f1-score   support

     damaged       0.89      0.91      0.90     30000
        good       0.50      0.45      0.48      6000

    accuracy                           0.83     36000
   macro avg       0.70      0.68      0.69     36000
weighted avg       0.83      0.83      0.83     36000
```

After changing to 64x64 images and using higher batch sizes, the f1 score jumped to 0.70. Also, training time increased because of the higher image sizes.

```
F1 (binary): 0.7056530214424951
              precision    recall  f1-score   support

     damaged       0.94      0.93      0.94     30000
        good       0.69      0.72      0.71      6000

    accuracy                           0.90     36000
   macro avg       0.82      0.83      0.82     36000
weighted avg       0.90      0.90      0.90     36000
```

Adding augmentation and balancing the data imbalance increased the f1 score for detecting good samples by artificially creating more samples, and raised our overall f1 score to 0.74.

```
563/563 ━━━━━━━━━━━━━━━━━━━━ 9s 15ms/step
F1 (binary): 0.7420494699646644
              precision    recall  f1-score   support

     damaged       0.97      0.92      0.94     30000
        good       0.66      0.84      0.74      6000

    accuracy                           0.90     36000
   macro avg       0.82      0.88      0.84     36000
weighted avg       0.92      0.90      0.91     36000
```

3.3 Discussion

Our hypothesis was supported because our first CNN got a very high accuracy but had problems recognizing the minority “good” class because of the dataset imbalance. Using augmentation and higher resolution imaging improved the accuracy for detecting good samples successfully.

4. Related Work

"A Lightweight Approach to Comprehensive Fabric Anomaly Detection Modeling"
In this paper, they address the high computational cost of deep-learning models for fabric defect detection. Their method, GH-YOLOx, uses lightweight modules, pruning, and knowledge distillation to improve speed and reduce parameters while maintaining accuracy. This approach could be worth looking into and comparing against what we're already doing, and we shouldn't immediately assume that our approach will be superior.

"Textile Anomaly Detection: Evaluation of the State-of-the-Art for Automated Quality Inspection of Carpet"
This study evaluates state-of-the-art unsupervised anomaly detection methods for carpet inspection, comparing models like student–teacher and generative approaches. They found student–teacher networks most effective, especially with multi-texture training. Our project could combine student–teacher methods with lightweight architectures for real-time use, and emphasize low false-alarm rates through cost-sensitive optimization.

5. Code and Dataset

The jupyter notebook can be found under this GitHub repo:
https://github.com/Chuck953/MachineLearning-UVM

To reproduce, download following dataset and extract the zip file:
https://www.kaggle.com/datasets/belkhirnacim/textiledefectdetection

Then edit "path_to_data" variable to the extracted folder. If the path is correct, the code should run just fine.

6. Conclusion

Our base model started off with promising results, with precision and recall for damaged images being approximately 90%, and a 90% F1-Score. Performance for detecting good images was much more poor, averaging around 50% for precision, 45% for recall, and an F1-Score of 48%.

The use of bigger images and augmentation successfully raised the F1-Score to 74%, greatly increasing the accuracy of our model. The training time increased due to the higher training complexity, but the sharp rise in accuracy is very much worth it.

With the addition of transfer learning (based off of the MobileNetV2 model), we saw minor but consistent improvements across the board in regards to precision, recall, F1-Score, and accuracy for all image types. While the improvements weren't major, the MobileNetV2 is a smaller model, and only minor improvements were expected anyways. With perhaps a larger more complex model, we may have seen even greater improvements, but in the interest of training time, we decided to stick with MobileNetV2. This is, however, something to consider in the future.

Our work will serve as a feasibility study to show how textiles can be detected by using different machine learning algorithms and designs. It also proves the importance of using higher detailed imaging, the efficiency of segmentation and offers a direct comparison using transfer learning. By further improving the existing algorithm, textiles could potentially be detected fully automatically, eliminating the existing bottleneck in production and greatly raising efficiency in the industry.

Bilbiography

Cui, S., Liu, W., & Li, M. (2025). A Lightweight Approach to Comprehensive Fabric Anomaly Detection Modeling. Sensors, 25(7), 2038. https://doi.org/10.3390/s25072038

Forsberg, B (2024). Textile Anomaly Detection: Evaluation of the State-of-the-Art for Automated Quality Inspection of Carpet. https://www.researchgate.net/publication/382638594_Textile_Anomaly_Detection_Evaluation_of_the_State-of-the-Art_for_Automated_Quality_Inspection_of_Carpet

Danish, M (2022). Textile Defect Detection: Binary Classification. Kaggle. https://www.kaggle.com/code/muhmddanish/textile-defect-detection-binary-classification#Textile-Defect-Detection-Project